

JavaScript Interview Questions & Answers

Completed Interview Practice — Questions 1–43

Focus: high-priority JavaScript interview concepts covered during the practice session. Answers are written in interview-ready form and include the key corrections discussed.

1. What is the JavaScript Engine?

Answer: A JavaScript engine is the program that parses and executes JavaScript code. It converts JavaScript code into executable instructions and manages execution-related concepts such as execution contexts, the call stack, and memory. Examples include V8, SpiderMonkey, and JavaScriptCore.

2. What is the Call Stack?

Answer: The Call Stack keeps track of the execution contexts and functions currently being executed. When a function is called, its execution context is pushed onto the stack; when it finishes, it is removed. The stack follows LIFO (Last In, First Out).

3. What is an Execution Context?

Answer: An execution context is the environment in which JavaScript code is executed. A function execution context is created when a function is called. Common types discussed are the Global Execution Context and Function Execution Context.

4. What is the Global Execution Context?

Answer: The Global Execution Context is created when JavaScript starts executing a program. It provides the environment for top-level/global code, including global variables, functions, and the global `this` value.

5. What is a Function Execution Context?

Answer: A Function Execution Context is created whenever a function is called. It contains the environment needed for that particular function execution, including parameters, local variables, scope information, and `this`.

6. Call Stack vs Execution Context

Answer: An execution context is the environment in which code executes. The Call Stack is the LIFO data structure used to keep track of currently active execution contexts. Context = where/how code executes; Stack = which execution is currently active.

7. Call Stack vs Heap

Answer: The Call Stack tracks execution contexts and active function calls. The Heap is a region of memory used for dynamically allocated data, such as objects and other values managed by the JavaScript engine.

8. Synchronous vs Asynchronous JavaScript

Answer: Synchronous JavaScript executes code sequentially, with each operation finishing before the next one starts. Asynchronous operations can complete later without blocking the main JavaScript execution, allowing

other work to continue.

9. How can JavaScript handle asynchronous operations if it is single-threaded?

Answer: JavaScript itself executes on a single main thread, while the browser provides Web APIs for operations such as timers, networking, and DOM events. When an asynchronous operation completes, its callback is placed into an appropriate queue. The Event Loop coordinates when queued callbacks can run on the Call Stack.

10. What is the Event Loop?

Answer: The Event Loop continuously monitors the Call Stack and the queues. When the current JavaScript execution finishes and the Call Stack is empty, microtasks are processed before the next task is taken from the Task Queue. This coordination enables asynchronous behavior.

11. Microtask Queue vs Task Queue

Answer: The Microtask Queue contains jobs such as Promise callbacks and `queueMicrotask()`. The Task Queue (often called the Callback/Macrotask Queue) contains tasks such as `setTimeout` callbacks and DOM events. Microtasks are processed before the next task.

12. Output: A, setTimeout, Promise, D

Answer: For `console.log('A');` `setTimeout(...'B'...);` `Promise.resolve().then(...'C'...);` `console.log('D');` the output is A, D, C, B. Synchronous code runs first, then microtasks, then the task callback.

13. What are Web APIs?

Answer: Web APIs are browser-provided APIs that give JavaScript access to browser features such as timers, DOM manipulation, networking, events, and storage. They are provided by the browser environment, not by the JavaScript language itself. Not every Web API is asynchronous.

14. Is setTimeout part of JavaScript or the browser?

Answer: `setTimeout` is a browser Web API. The browser handles the timer. When the timer has finished, its callback is placed in the Task Queue. When the Call Stack is empty and higher-priority microtasks are handled, the Event Loop allows that callback to execute.

15. What is a Promise?

Answer: A Promise is an object representing the eventual completion or failure of an asynchronous operation and its resulting value. Its three states are Pending, Fulfilled, and Rejected.

16. Why does a resolved Promise callback run after synchronous code?

Answer: Calling `resolve()` fulfills the Promise, but callbacks attached with `.then()` are scheduled as microtasks. Therefore synchronous code already on the Call Stack finishes first; then the microtask runs.

17. What do then(), catch(), and finally() do?

Answer: `.then()` handles a fulfilled Promise and returns a new Promise. `.catch()` handles a rejected Promise/error. `.finally()` runs after the Promise settles regardless of whether it was fulfilled or rejected.

18. `async/await` vs Promises

Answer: `async/await` is syntax built on top of Promises. It provides a cleaner way to write Promise-based asynchronous code. An `async` function always returns a Promise.

19. Does `await` block the JavaScript thread?

Answer: No. `await` pauses the execution of the current `async` function until the awaited Promise settles. It does not block the entire JavaScript thread. `fetch()` is also non-blocking; the browser handles the network operation.

20. Output: `async` function with `await Promise.resolve()`

Answer: For a function that logs A, awaits `Promise.resolve()`, then logs B, followed by `console.log('C')`, the output is A, C, B. `await` pauses the current `async` function and schedules its continuation as a microtask.

21. What is Hoisting?

Answer: Hoisting describes how declarations are processed before the corresponding code executes. It does not literally mean that JavaScript moves code to the top. With `var`, the declaration is available and initialized to `undefined` before its assignment.

22. What is the Temporal Dead Zone?

Answer: The Temporal Dead Zone is the period from entering a scope until execution reaches a `let` or `const` declaration. The variable is hoisted but remains uninitialized, so accessing it during the TDZ causes a `ReferenceError`.

23. Scope example: global a and function-local b

Answer: A function can access variables from its own scope and outer scopes, but the outer scope cannot access variables declared inside the function. Therefore a is accessible inside `test()`, but b is not accessible outside `test()`, causing a `ReferenceError`.

24. What is Lexical Scope?

Answer: Lexical scope means a function's accessible variables are determined by where the function is written/defined, not where it is called. A function can access variables from its own scope and outer lexical scopes through the scope chain.

25. Closure output and meaning

Answer: For `outer()` creating `count = 0` and returning `inner()`, `const counter = outer(); counter(); counter();` outputs 1 and 2. `inner` retains access to the lexical environment containing `count` even after `outer()` has finished. A closure is a function plus its remembered outer environment.

26. Practical uses of closures

Answer: Closures are commonly used for data privacy/encapsulation, function factories, and callbacks that need access to variables from their surrounding scope.

27. var vs let vs const

Answer: var is function-scoped and allows redeclaration and reassignment. let is block-scoped, does not allow redeclaration in the same scope, and allows reassignment. const is block-scoped, does not allow redeclaration or reassignment. let and const have a Temporal Dead Zone.

28. == vs ===

Answer: == performs loose equality and may perform type coercion before comparison. === performs strict equality and compares type and value without coercion. Example: 5 == '5' is true, while 5 === '5' is false.

29. Type coercion outputs

Answer: 1 + '2' → '12' because + with a string performs concatenation. 1 - '2' → -1 because numeric operators convert '2' to a number. true + 1 → 2 because true becomes 1. null + 1 → 1 because null becomes 0 in numeric conversion.

30. What is this?

Answer: The value of this is generally determined by how a regular function is called. For a method call such as user.greet(), this refers to user. It is not determined simply by where the function is written.

31. Regular function this vs arrow function this

Answer: Regular functions have their own this binding determined by the call site. Arrow functions do not have their own this; they lexically inherit this from the surrounding scope.

32. Regular vs arrow method output

Answer: For user.regular(), this refers to user, so this.name is 'Kiran'. For user.arrow(), the arrow function does not get this from user; it inherits this from its surrounding context, so this.name is not 'Kiran' (commonly undefined in the discussed browser context).

33. call(), apply(), bind()

Answer: All three can explicitly control a function's this value. call() invokes immediately with individual arguments. apply() invokes immediately with arguments supplied as an array/array-like object. bind() does not invoke immediately; it returns a new function with this bound.

34. What is a Prototype and Prototype Chain?

Answer: A prototype is an object from which another object can inherit properties and methods. The prototype chain is the sequence of objects JavaScript searches when a property or method is not found on the current object.

35. How does property lookup work?

Answer: JavaScript first looks for the property on the current object. If it is not found, it searches the object's prototype, then that prototype's prototype, continuing until the property is found or the chain reaches null.

36. `__proto__` vs `Constructor.prototype`

Answer: `__proto__` refers to an object's prototype link. `Constructor.prototype` is a property on constructor functions that is used as the prototype for instances created with `new`. Modern code generally prefers `Object.getPrototypeOf()` and `Object.setPrototypeOf()` over direct `__proto__` usage.

37. What happens with `new`?

Answer: The `new` operator creates a new object, links it to the constructor's prototype, calls the constructor with this referring to the new object, and returns the resulting object (unless the constructor explicitly returns another applicable object).

38. Shallow Copy vs Deep Copy

Answer: A shallow copy creates a new top-level object but keeps references to nested objects. A deep copy creates independent nested objects as well. Example: `{...original}` is shallow; `structuredClone(original)` can make a deep copy for supported data.

39. Event Bubbling

Answer: Event bubbling is propagation from the target element upward through its ancestors. Clicking a button inside a div can trigger the button handler first and then the div handler.

40. Event Capturing

Answer: Event capturing is the phase where an event travels from the top of the DOM tree down toward the target. The overall flow is capturing phase → target phase → bubbling phase.

41. Event Delegation

Answer: Event delegation is a technique where a parent handles events for child elements using event bubbling. The parent can identify the actual element that triggered the event using `event.target`. It reduces the need for many individual listeners and works well with dynamically added elements.

42. Debouncing

Answer: Debouncing delays function execution until a specified time has passed since the last event. For a search box, repeated keystrokes reset the timer, and the API call happens once after the user stops typing.

43. Throttling

Answer: Throttling limits a function to execute at most once during a specified time interval. It is useful for continuous events such as scrolling. Memory trick: Debounce = wait until you stop; Throttle = keep going but run periodically.